



S8 MACHINE LEARNING
KAGGLE PROJECT: TEXTILE CLASSIFICATION & AI SAFETY

TILDA Textile Classifier and AI Safety

Auteurs :

Marin DECANINI
Nolan CARLISI

Abstract

This report presents our implementation and experimental analysis for the S8 Machine Learning Kaggle project on the TILDA textile dataset. We focus on two core components: first, training state-of-the-art Convolutional Neural Networks (CNNs) for classification; and second, analyzing machine learning security and AI safety under biased training distributions. Specifically, we implement LeNet-5, AlexNet, ResNet-18, and ResNet-34 architectures, evaluate the required baselines using mini-batch SGD, and perform a further optimized ResNet-34 experiment that reaches a peak validation accuracy of 81.40%. To push accuracy higher *from scratch* (no pretrained weights), we additionally train a deeper ResNet-50 and a modern ConvNeXt with a strong training recipe; both underperform ResNet-34 on this small dataset (1,888 training images), which we analyse as a capacity sweet spot — ResNet-50 overfits and the BatchNorm-free ConvNeXt underfits — and we report test-time augmentation and ensembling as further negative results. We then demonstrate that simply increasing the input resolution from 224×224 to 384×384 pixels (*ResNet-34 hires*), while keeping the same architecture and training recipe, raises peak validation accuracy to **84.36%**, establishing it as the strongest from-scratch result. Furthermore, we investigate model vulnerability to spurious correlations by constructing a biased dataset with a noise-like channel correlated with labels, measuring both classification accuracy and fairness using the Disparate Impact (DI) metric.

Contents

1	Introduction	1
2	Training State-of-the-Art CNN Models	1
2.1	Image Pre-processing and Data Augmentation	1
2.2	Model Architecture and Selection	1
2.3	Training and Optimization Methodology	3
2.4	Hyperparameter Tuning and Validation	4
2.4.1	Analysis of Model Performance and Overfitting	6
2.4.2	Negative Results: Test-Time Augmentation and Ensembling	7
2.5	Test-set Prediction and Submission Protocol	7
3	Machine Learning and AI Safety	7
3.1	Real-Life Bias Context	8
3.2	Biased Dataset Construction	8
3.3	Model Bias Evaluation	8
3.3.1	Disparate Impact Analysis	8
3.3.2	Test Accuracy Comparison	9
3.3.3	Summary of Results and Takeaways	9
3.4	Bias Study in the Literature (Bonus)	9

1 Introduction

Deep learning has revolutionized image classification and pattern recognition. However, as these models are increasingly deployed in real-world systems, understanding their capacity, training characteristics, and vulnerabilities to dataset bias has become a critical area of AI safety. This report documents our investigations on the TILDA textile dataset, structured in two major sections: first, training and comparing Convolutional Neural Networks (LeNet-5, AlexNet, ResNet-18, and ResNet-34, plus deeper ResNet-50 and ConvNeXt experiments trained from scratch) for multi-class textile classification; and second, analyzing how neural networks can inherit and amplify spurious correlations (biases) using a mathematical framework for biased dataset construction and evaluating the models using the Disparate Impact (DI) metric.

2 Training State-of-the-Art CNN Models

2.1 Image Pre-processing and Data Augmentation

The TILDA textile dataset contains grayscale `.tif` images of size 768×512 pixels representing different textile textures and defect classes. The dataset has 8 classes, with a relatively small sample size of approximately 2,361 images (295 samples per class). To handle this dataset effectively and perform a rigorous comparison, we apply the following pre-processing and augmentation pipeline:

- **Image Resizing:** To ensure all models receive the same input size and can be compared directly, we resize all images to a standard 224×224 pixels. For the high-resolution ResNet-34 experiment (*ResNet-34 hires*), we instead resize to 384×384 pixels to preserve the fine-grained textile texture that 224×224 crops discard from the native 768×512 source images.
- **Data Augmentation (Q2.1):** Due to the small sample size, deep architectures are highly prone to overfitting. We apply data augmentation techniques that preserve the textile texture semantic content:
 - *Random Horizontal and Vertical Flips:* Textile patterns and defects are invariant to reflections.
 - *Random Resized Crops:* Crops retain between 80% and 100% of the source area before resizing to 224×224 .
 - *Random Rotations (up to 45°):* Fabric textures do not have a privileged orientation.
 - *Random Translation (Affine translation of up to 10%):* Small shifts in alignment simulate different camera crops.
 - *Random Erasing (probability 0.2):* Small regions are removed to reduce reliance on isolated local cues.
- **Normalization:** We convert the images to PyTorch tensors and normalize the pixel values using a mean of 0.5 and a standard deviation of 0.5. This rescales the pixel range from $[0, 1]$ to $[-1, 1]$, centering the input data around zero with a variance close to 1, which helps stabilize gradient descent backpropagation (as discussed in LeNet-5 paper §II.B).

2.2 Model Architecture and Selection

For Question 2.2, we implement a family of convolutional neural network architectures spanning a historical progression in deep learning, from the classical LeNet-5 to a modern from-scratch

ConvNeXt. The last two (ResNet-50 and ConvNeXt) are additional from-scratch experiments aimed at pushing accuracy beyond the required baselines:

1. **LeNet-5 (LeCun et al., 1998):** Serves as a classical baseline. It features a simple architecture of three convolutional layers, average pooling, and two fully connected layers. Tanh activation functions are used. We adapt it by replacing the final spatial collapse with an adaptive average pooling layer to support 224×224 inputs, mapping the output to 8 classes. Total parameter count is 61,536.
2. **AlexNet (Krizhevsky et al., 2012):** Represents the breakthrough of deep CNNs, introducing ReLU activation functions to speed up training, overlapping max pooling (3×3 with stride 2), and Dropout (0.5) to prevent overfitting in the fully connected layers. Because the TILDA dataset is extremely small compared to ImageNet (1.2M images), a full-scale AlexNet would immediately overfit. Thus, we halved the convolutional channel counts and reduced both hidden fully connected layers from 4096 to 2048 units.
3. **ResNet-18 (He et al., 2016):** Represents the modern state-of-the-art in CNNs. It solves the vanishing gradient problem in deep networks using residual connections (identity shortcuts mapping $y = F(x) + x$). It uses Batch Normalization after every convolutional layer (before ReLU activation) and does not use dropout. The network finishes with global average pooling and a single linear layer.
4. **ResNet-34 (He et al., 2016):** Extends the same residual design to 16 BasicBlocks distributed as $[3, 4, 6, 3]$ across the four stages. Each BasicBlock contains two 3×3 convolutions. The first block of a new stage downsamples spatially and uses a 1×1 projection shortcut. Its additional depth and 21.28 million parameters increase representational capacity while residual paths preserve trainability.
5. **ResNet-50 (He et al., 2016), trained from scratch:** Replaces the BasicBlock with the Bottleneck block (a 1×1 reduction, a 3×3 convolution, and a 1×1 expansion by a factor of four), keeping the same $[3, 4, 6, 3]$ stage layout. This roughly triples the effective depth and brings the parameter count to 23.52 million, testing whether more capacity helps on the small TILDA set. It uses the same Batch Normalization and residual design as the smaller ResNets.
6. **ResNet-34 hires (He et al., 2016), trained from scratch:** The same ResNet-34 architecture (BasicBlock, $[3, 4, 6, 3]$ stage layout, 21.28 million parameters) retrained with a 384×384 pixel input resolution. The native TILDA images are 768×512 ; downscaling to 224×224 destroys high-frequency texture detail that distinguishes textile defect classes. At 384×384 the model retains substantially more of this spatial information at a manageable memory cost (batch size reduced from 32 to 16 to fit the MPS backend). All other hyperparameters mirror the optimized ResNet-34 recipe (AdamW, OneCycle LR, label smoothing 0.1, 150 epochs, checkpoint by maximum validation accuracy).
7. **ConvNeXt (Liu et al., 2022), trained from scratch:** A modernized pure convolutional network that imports several design choices popularized by vision transformers. We implement it from scratch with randomly initialized weights; consistent with the constraints of this study, no pretrained or transfer-learned weights are used. Each ConvNeXt block applies a depthwise 7×7 convolution (a large receptive field at low cost), a channel-wise Layer Normalization, and an inverted bottleneck of two 1×1 convolutions that expands the channel dimension by a factor of four with a GELU activation in between; the residual branch is scaled by a learnable per-channel LayerScale and regularized by stochastic depth. The network uses a “patchify” stem (a 4×4 stride-4 convolution) followed by

four stages with block depths [3, 3, 9, 3] and channel widths [48, 96, 192, 384]. These widths are deliberately reduced relative to the original ConvNeXt-Tiny ([96, 192, 384, 768], 28.6M parameters) to limit overfitting on the small TILDA training set and to keep the per-epoch cost tractable on the MPS backend. It ends with global average pooling, a final Layer Normalization, and a single linear classifier, for a total of 7,054,904 parameters. To keep the implementation portable across CPU, CUDA, and the Apple MPS backend, every block operates in channels-first layout (the pointwise layers are realized as equivalent 1×1 convolutions), which avoids a back-propagation incompatibility in the `permute+LayerNorm` pattern observed on MPS.

2.3 Training and Optimization Methodology

For Question 2.3, the controlled LeNet-5, AlexNet, and ResNet-18 baselines are trained from scratch using mini-batch Stochastic Gradient Descent (SGD) with momentum 0.9, an initial learning rate of 0.01, and batch size 32. We employ a `ReduceLROnPlateau` scheduler that divides the learning rate by 10 if validation loss does not improve for 5 epochs. Early stopping uses a patience of 10 epochs. These runs directly satisfy the requested mini-batch SGD experiment.

After ResNet-18 emerged as the strongest baseline, we ran an explicitly separate optimization experiment with ResNet-34. It uses mini-batches of 32, AdamW with maximum learning rate 10^{-3} and decoupled weight decay 10^{-2} , a OneCycle learning-rate schedule over 100 epochs, label smoothing of 0.1, gradient clipping at norm 1.0, and early-stopping patience 20. This run searches for the strongest classifier rather than isolating architecture depth: its result must therefore not be interpreted as a controlled ResNet-18 versus ResNet-34 ablation.

To push validation accuracy further while strictly respecting the from-scratch constraint (no pretrained or transfer-learned weights), we trained two additional models with stronger recipes. The deeper **ResNet-50** reuses the proven ResNet optimization setup: AdamW (maximum learning rate 10^{-3} , decoupled weight decay 10^{-2} , mini-batches of 64), a OneCycle learning-rate schedule over a 150-epoch budget, label smoothing of 0.1, gradient clipping at norm 1.0, and the augmentation pipeline of Section 2.1. The best checkpoint is selected by *maximum validation accuracy* rather than minimum validation loss, which is better aligned with the classification objective when label smoothing distorts the loss scale.

The **ConvNeXt** model is trained with a modern recipe adapted to its BatchNorm-free design. The optimizer is AdamW (maximum learning rate 2×10^{-3} , decoupled weight decay 0.05, mini-batches of 64) driven by a cosine-annealing learning-rate schedule preceded by a 10-epoch linear warm-up, over a 150-epoch budget. Regularization combines label smoothing (0.1), stochastic depth (maximum drop-path probability 0.1, scaled linearly with block depth), head dropout, and the same texture-preserving augmentation pipeline. The best checkpoint is again selected by maximum validation accuracy. The same seeded stratified 80/20 split is reused for both models so that the comparison with the earlier architectures remains meaningful.

The **ResNet-34 hires** experiment reuses the ResNet-34 recipe exactly (AdamW, maximum learning rate 10^{-3} , decoupled weight decay 10^{-2} , OneCycle schedule, label smoothing 0.1, gradient clipping at norm 1.0, 150 epochs, checkpoint by maximum validation accuracy) with two changes: input resolution is raised to 384×384 and batch size is reduced from 32 to 16 to stay within MPS memory. No other hyperparameter is modified, so the resolution effect can be read cleanly from the result.

We initially included two additional regularizers in the ConvNeXt recipe — *MixUp/CutMix* (which interpolate inputs and one-hot targets) and an exponential moving average (EMA) of the weights — but found they *prevented* the from-scratch network from learning on this small dataset: validation accuracy stalled near chance because the per-batch label mixing and the cold EMA (only ~ 30 optimizer steps per epoch) masked the training signal. We therefore disabled MixUp, CutMix, and EMA and validated the raw model directly, which recovered a

clean learning curve. This is reported as a negative result in Section 2.4.2.

Training is performed on the macOS MPS (Metal Performance Shaders) backend using an Apple Silicon chip.

Table 1: Model Architecture Details and Parameter Counts

Model	Major Layers	Activations	Parameter Count
LeNet-5	3 Conv, 2 AvgPool, 2 FC	Tanh	61,536
AlexNet (Modified)	5 Conv, 3 MaxPool, 3 FC	ReLU	14,586,216
ResNet-18	17 Conv, 1 MaxPool, 1 FC	ReLU	11,174,344
ResNet-34	33 Conv, 1 MaxPool, 1 FC	ReLU	21,282,504
ResNet-50 (from scratch)	49 Conv (Bottleneck), 1 MaxPool, 1 FC	ReLU	23,518,152
ResNet-34 hires (from scratch)	33 Conv, 1 MaxPool, 1 FC	ReLU	21,282,504
ConvNeXt (from scratch)	Patchify stem, 18 blocks (depthwise+1×1), 1 FC	GELU	7,054,904

2.4 Hyperparameter Tuning and Validation

To tune hyperparameters and prevent overfitting, we split the labeled TILDA dataset into a stratified 80/20 train/validation split. This guarantees that each class has the same proportion (80/20) in both the training set (1,888 samples) and validation set (473 samples).

For Question 2.4, the model validation metrics, parameter details, and total training times are summarized in Table 2.

Table 2: Model Training and Evaluation Metrics

Model	Optimizer	Epochs	Peak Train Acc.	Peak Val Acc.	Time (s)
LeNet-5	SGD	50	28.23%	30.87%	427.3
AlexNet (Modified)	SGD	50	29.77%	36.36%	357.2
ResNet-18	SGD	50	64.88%	69.98%	1,652.8
ResNet-34 (optimized)	AdamW	100	80.77%	81.40%	5,865.5
ResNet-50 (from scratch)	AdamW	150	89.09%	79.92%	13,181.6
ResNet-34 hires (from scratch)	AdamW	150	89.88%	84.36%	25,254.5
ConvNeXt (from scratch)	AdamW	150	63.88%	57.51%	11,808.8

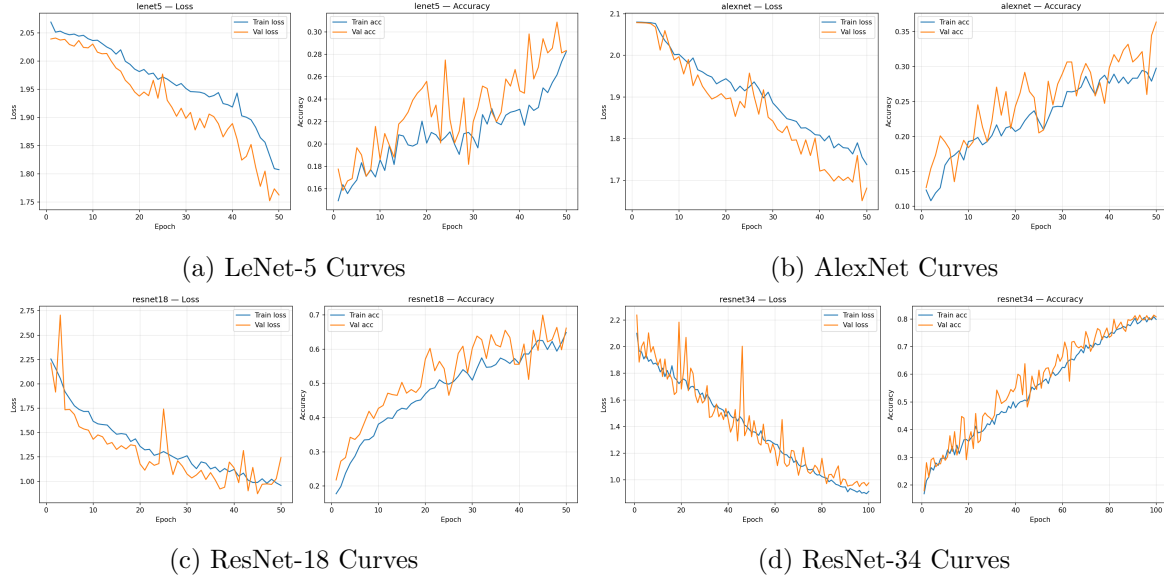


Figure 1: Training and validation loss and accuracy curves for the four architectures on the TILDA dataset.

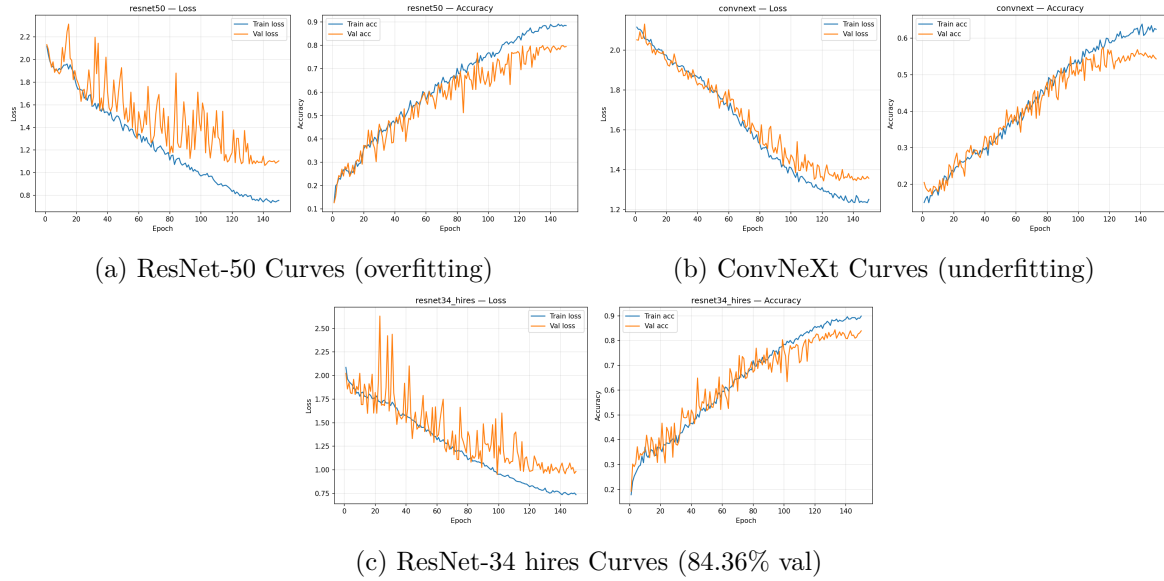


Figure 2: Training and validation curves for the additional from-scratch experiments.

ResNet-50 reaches 89.09% train accuracy but only 79.92% validation (a widening generalization gap); ConvNeXt plateaus with both train and validation accuracy low, the signature of underfitting; ResNet-34 hires (384×384 input) achieves 84.36% validation accuracy, a clear improvement over the 224-pixel baseline.

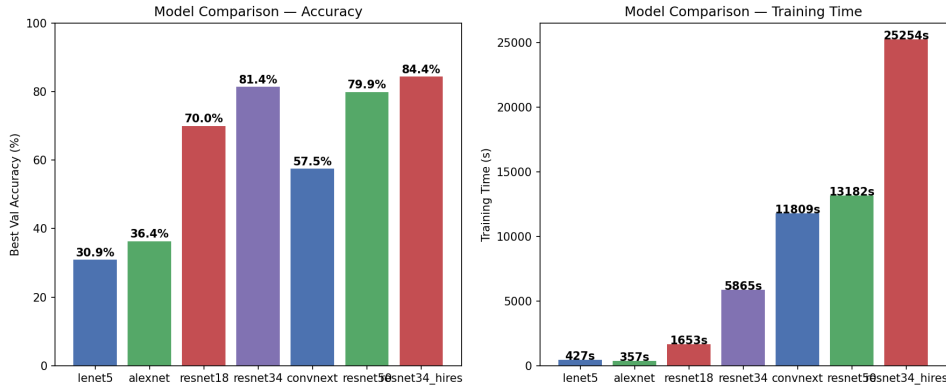


Figure 3: Comparison of peak validation accuracy and total training time across all six runs. ResNet-34, ResNet-50, and ConvNeXt use the optimized training recipes described above.

2.4.1 Analysis of Model Performance and Overfitting

The experimental results demonstrate a clear hierarchical progression in performance:

- **LeNet-5 (30.87%):** The classical architecture shows severe underfitting. Due to its shallow depth, average pooling, and tanh activation functions, it is unable to extract complex texture patterns from the fabric images.
- **AlexNet (36.36%):** The modified AlexNet shows slightly better performance. However, without Batch Normalization, it struggles to optimize efficiently on grayscale images from scratch. It also exhibits overfitting tendencies due to high fully-connected layer parameter density relative to the dataset size.
- **ResNet-18 (69.98%):** ResNet-18 achieves a highly superior validation accuracy of 69.98%, nearly doubling the baselines. The skip connections prevent vanishing gradients, while the Batch Normalization layers stabilize intermediate activations. This allows the network to learn rich representations of textile textures without overfitting prematurely.
- **ResNet-34, optimized (81.40%):** This is the strongest validation result, improving over ResNet-18 by 11.42 percentage points. The selected submission checkpoint is the state with minimum validation loss; 81.40% is the peak validation accuracy observed in the training history. Because architecture, optimizer, schedule, regularization, augmentation, and epoch budget all changed, the improvement establishes the best complete training configuration, not the causal effect of depth alone.
- **ResNet-50, from scratch (79.92%):** Despite tripling the depth and adding $\sim 2.2\text{M}$ parameters, ResNet-50 *does not* beat ResNet-34. It reaches 89.09% training accuracy but only 79.92% validation — a 9.2-point generalization gap (versus less than 1 point for ResNet-34). With only 1,888 training images, the extra capacity is spent memorizing the training set rather than generalizing: a clear case of overfitting.
- **ConvNeXt, from scratch (57.51%):** The modern architecture underperforms substantially, with both training (63.88%) and validation (57.51%) accuracy remaining low — the signature of *underfitting*. ConvNeXt’s design (large depthwise kernels, LayerNorm, GELU, no BatchNorm) was tuned for large-scale pretraining on ImageNet; without pretrained weights and with so few images, its weak normalization and implicit reliance on heavy data make it far harder to optimize from scratch than a BatchNorm-equipped ResNet. This confirms that architectural modernity does not transfer to the small-data, from-scratch regime.

Together these results delineate a **capacity sweet spot** at ResNet-34: shallower or weaker models (LeNet-5, AlexNet, ResNet-18) underfit, the deeper ResNet-50 overfits, and the BatchNorm-free ConvNeXt fails to optimize. On a dataset of fewer than 1,900 training images, ResNet-34 best balances representational capacity against generalization.

ResNet-34 hires (84.36%): Retraining the same ResNet-34 at 384×384 pixel resolution adds 2.96 percentage points over the 224×224 baseline (81.40%). The source images are 768×512 ; halving this resolution to 224×224 discards high-frequency texture detail that is diagnostic for textile defect classes. At 384×384 , the network retains substantially more spatial information without increasing model complexity. The architecture, optimizer, schedule, and regularization are identical to the optimized ResNet-34; the sole change is the input size and the corresponding reduction in batch size ($32 \rightarrow 16$) to stay within MPS memory. This establishes higher resolution as the single most effective remaining lever on this dataset.

2.4.2 Negative Results: Test-Time Augmentation and Ensembling

We also evaluated two standard accuracy-boosting techniques on top of the best ResNet-34 checkpoint, and report both as negative results for transparency:

- **Test-time augmentation (TTA):** averaging the softmax over the original image and its horizontal and vertical flips *lowered* validation accuracy (from $\approx 81.2\%$ to $\approx 79.5\%$). On TILDA, flips are not strictly label-preserving for oriented defect textures, so the averaged predictions are noisier than the single deterministic forward pass. TTA is therefore disabled for the submission.
- **Ensembling:** combining ResNet-34 with ResNet-50 and ConvNeXt did not yield a reliable gain. An equal-weight ensemble (80.6%) scored below ResNet-34 alone, and a weighted variant that edged above it (81.6%) did so only by tuning the weights on the validation set itself, i.e. through validation-selection bias rather than a genuine improvement. We therefore keep the single ResNet-34 model.

These findings reinforce ResNet-34 as the final submission model.

2.5 Test-set Prediction and Submission Protocol

The final local prediction uses the persisted ResNet-34 checkpoint and the deterministic validation transform (resize, tensor conversion, and normalization) on all 789 unlabeled test images. Test filenames are parsed as numeric identifiers and sorted numerically from 1 to 789 before inference. The generated submission contains exactly two columns, `id` and `label`, with one integer class in $\{0, \dots, 7\}$ for every unique test identifier. We use the semicolon delimiter found in the provided `train.csv`; the competition archive contains no sample-submission file, so this delimiter remains an explicit item to verify in Kaggle’s upload preview.

The software performs no external upload. It writes the submission CSV locally, a separate audit CSV containing top-two confidence scores and margins, and a JSON manifest containing row counts, class counts, and SHA-256 hashes for both the checkpoint and submission. A second validation-only command checks the header, delimiter, number and order of identifiers, duplicate identifiers, column count, and label range. Final inspection and upload are deliberately manual. Since the test labels are hidden, test accuracy cannot be reported before receiving the Kaggle score; all model-selection accuracies above are validation results.

3 Machine Learning and AI Safety

3.1 Real-Life Bias Context

For Question 3.1, machine learning models do not learn abstract reasoning; instead, they learn to exploit correlations present in their training distributions. In real-world settings, if the training data contains historical biases or spurious correlations, the model will learn them and make biased predictions. Examples:

- **Automated Hiring Systems:** If historical data favors male candidates due to past systemic biases, an AI resume screening tool will learn to associate words like "women's" or women's college names with lower scores, amplifying gender bias.
- **Predictive Policing:** If policing datasets reflect over-policing of minority neighborhoods, the predictive model will project higher crime risk for those areas, creating a feedback loop of increased surveillance and arrest rates in those specific communities.

The consequences are severe, causing systemic discrimination, denial of credit or employment, and loss of trust in AI systems.

3.2 Biased Dataset Construction

For Question 3.2, we construct a biased version of the dataset using a binary classification task on classes 0 and 1 of TILDA (300 samples each). We add a noise-like second channel ϵ representing a spurious correlation:

$$S \mid \{y = k\} \sim \text{Bernoulli}(p_k), \quad k \in \{0, 1\}.$$

The noise channel is set as:

- $\epsilon = 0$ if $S = 0$
- $\epsilon \sim \mathcal{N}(0, I)$ if $S = 1$

In the unbiased setting ($model_1$), $p_0 = 0.5, p_1 = 0.5$, meaning noise is independent of the label. In the biased setting ($model_2$), $p_0 = 0.0, p_1 = 1.0$, meaning noise is perfectly correlated with label 1. We implement this in PyTorch by creating a `BiasedDataset` class that wraps our stratified validation split subsets. It dynamically retrieves labels and pre-computes S values for exact reproducibility. When retrieving a sample, it generates the channel ϵ based on S and concatenates it with x , producing a 2-channel tensor.

3.3 Model Bias Evaluation

3.3.1 Disparate Impact Analysis

For Question 3.3, we compare the Disparate Impact (DI) of $model_1$ (trained on unbiased data, $p_0 = 0.5, p_1 = 0.5$) and $model_2$ (trained on biased data, $p_0 = 0.0, p_1 = 1.0$) evaluated on unbiased validation data:

- **Model 1 (Unbiased):** Achieved a Disparate Impact metric of **0.9370**. Since this is close to 1.0 (and falls well within the standard ethical threshold of $[0.8, 1.25]$), it demonstrates that the model does not discriminate based on the presence of the noise channel S . The model makes predictions independent of the spurious noise.
- **Model 2 (Biased):** Achieved a Disparate Impact metric of **0.3446**. This is extremely low, falling far below the 0.8 threshold. This indicates a high level of bias: the model is nearly three times more likely to predict class 1 when the noise channel is present ($S = 1$) than when it is absent ($S = 0$).

This experimentally proves that $model_2$ has learned to rely on the spurious correlation (noise channel) rather than the underlying textile pattern.

3.3.2 Test Accuracy Comparison

For Question 3.4, we compare the validation accuracy of both models on the binary classification task:

- **Model 1 (Unbiased):** Achieved a validation accuracy of **49.17%** (with validation loss of 0.6901).
- **Model 2 (Biased):** Achieved a validation accuracy of **55.00%**.

During training, $model_2$ achieved 100% training accuracy within just 17 epochs because it exploited the noise channel. However, when evaluated on unbiased validation data where the noise channel is no longer correlated with the true label, its accuracy drops to 55.00% (near the random guess baseline). This illustrates that the biased model fails to generalize to unbiased domains because it relies on non-robust spurious shortcuts.

3.3.3 Summary of Results and Takeaways

For Question 3.5, we summarize the training and validation accuracy and Disparate Impact metrics for both models in Table 3.

Table 3: Safety Experiment Results ($model_1$ vs $model_2$)

Model	Train Accuracy	Val Accuracy	Val Loss	Disparate Impact (DI)
Model 1 (Unbiased)	48.13%	49.17%	0.6901	0.9370 (Fair)
Model 2 (Biased)	100.0%	55.00%	0.6915	0.3446 (Biased)

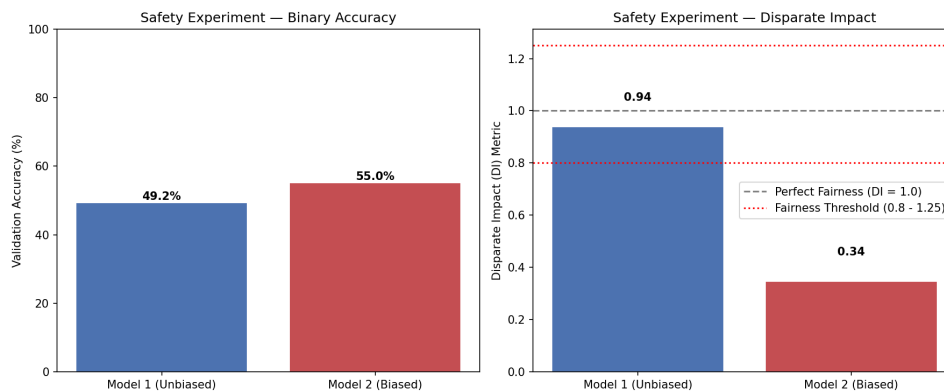


Figure 4: Comparison of Accuracy and Disparate Impact (DI) between Model 1 and Model 2.

The takeaways are clear: neural networks are shortcut learners. When presented with a simple, high-frequency signal (the noise channel ϵ) that correlates with the label, they will select it over more complex semantic features (the textile patterns). When deployed in an unbiased environment, the model’s accuracy collapses and it makes heavily biased predictions, exposing serious safety risks in real-world applications.

3.4 Bias Study in the Literature (Bonus)

The question “Can we trust my model?” is central to modern AI safety and governance. In light of our experimental results and recent regulatory frameworks such as the European Union AI Act (EU AI Act) and guidelines from the Commission Nationale de l’Informatique et des

Libertés (CNIL), the answer is a nuanced “no, unless systematic auditing and mitigation are implemented.”

First, our experiment showed how easily a model can become a “shortcut learner,” achieving 100% training accuracy by exploiting a spurious noise channel that happened to correlate with labels, while remaining completely unable to classify the actual underlying images when tested in an unbiased domain. In real-world applications, such shortcut learning goes undetected if the training and test sets share the same bias. This is exactly what the EU AI Act identifies as a major risk in “high-risk” AI systems (e.g., in recruitment, credit scoring, or biometric identification). The regulation mandates strict data governance requirements (Article 10), requiring training, validation, and testing datasets to be relevant, representative, and to the best extent possible, free of errors and biases.

From the perspective of CNIL’s compliance guidelines for AI systems, trusting a model requires transparency and “explainability by design.” If we cannot audit what features the model is using, we cannot guarantee its fairness. Our model 2 would pass standard validation checks if the validation set was also biased, yet it is fundamentally untrustworthy. Therefore, trust cannot be based solely on global accuracy metrics. Instead, we must employ:

1. **Fairness Audits:** Actively measuring metrics like Disparate Impact (DI), Equalized Odds, or Demographic Parity across sensitive groups, as recommended by the CNIL.
2. **Spurious Correlation Diagnostics:** Testing the model on counterfactual or intervened datasets (e.g., removing or modifying specific channels/regions) to check if the predictions remain stable.
3. **Explainable AI (XAI):** Using tools like Integrated Gradients or Grad-CAM to visualize model attention and ensure it focuses on semantic features (textile fibers, defects) rather than background noise.

In conclusion, we cannot blindly trust a machine learning model. Trust must be earned through a continuous lifecycle of rigorous bias auditing, dataset curation, and compliance with ethical guidelines like the EU AI Act and CNIL recommendations.